


# Il primo programma in C

Parte 2: uguaglianza, funzioni e librerie, input/output, conversioni

**Shahram Rahatlou**

Corso di Laboratorio di Calcolo, Canale Q-Z
Anno Accademico 2026-2027

SAPIENZA
UNIVERSITÀ DI ROMA

# Assegnazione = contro uguaglianza ==

*Un solo carattere in più, un significato molto diverso*

## `a = b;`

assegnazione: il valore di b va in a

| a | b |
|---|---|
| 2 → 3 | 3 |

## `b = a;`

il valore di a va in b

| a | b |
|---|---|
| 2 | 3 → 2 |

## `a == b`

domanda: a e b hanno lo stesso valore? **vero (1)** oppure **falso (0)**

`con a = 2 e b = 3:  a == b  è falso,  b == a  è falso`

# Vero e falso sono numeri

*Il risultato di un confronto si può salvare in una variabile int*

operatori.c

```
int x = 2, y = 3;
int decisione;

decisione = (x == y);    // falso: 0
decisione = (x != y);    // vero:  1
decisione = (x >  y);    // falso: 0
```

*i valori di partenza*

| x | y |
|---|---|
| 2 | 3 |

ogni riga sovrascrive decisione con il risultato del confronto

| decisione | decisione | decisione |
|---|---|---|
| 0 | 1 | 0 |
| dopo (x == y) | dopo (x != y) | dopo (x > y) |

falso: decisione vale 0

vero: decisione vale 1

# Funzioni: dati in ingresso, dati in uscita

*C è un linguaggio strutturato: funzioni e strutture dati*




`a = √3` → 3 è l'input, a è il valore di output

### main è una funzione

senza dati in ingresso, restituisce il risultato del programma al sistema operativo

### printf e sqrt

sono funzioni già pronte, che troviamo nelle librerie standard

# Usare le funzioni matematiche

*math.h dichiara le funzioni e le costanti della libreria matematica*

radice.c

```c
#include <math.h>
#include <stdio.h>

int main() {
    double x;
    x = sqrt(2.4);
    printf("radice di 2.4 = %lf\n", x);
}
```

### #include

istruzione per il precompilatore, non per la CPU

### math.h

header file: contiene le dichiarazioni di tutte le funzioni e costanti matematiche

### sqrt

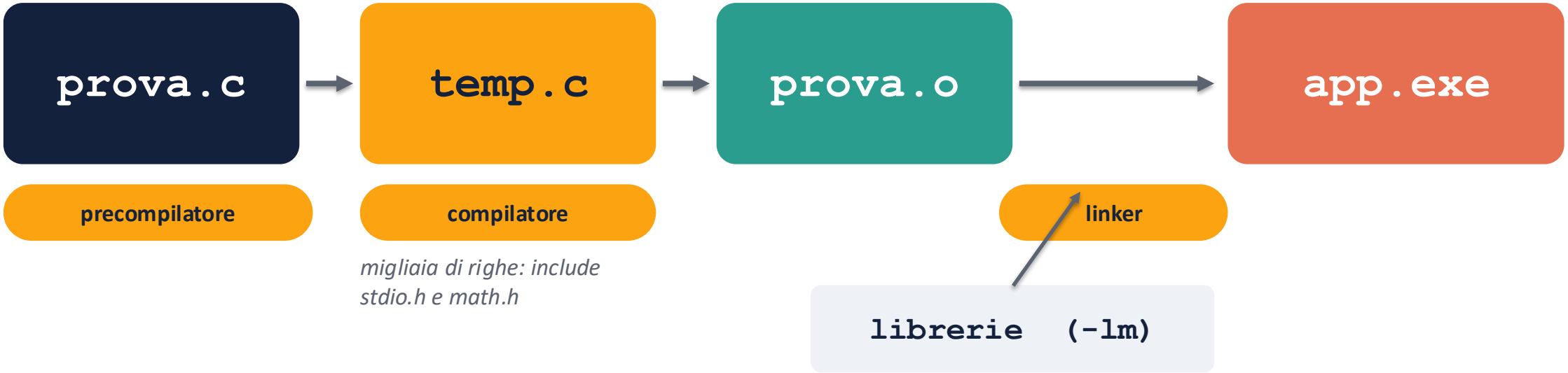
calcolo della radice quadrata

Terminale

```
$ gcc -o app.exe radice.c -lm
$ ./app.exe
radice di 2.4 = 1.549193
```

# Il preprocessore

*Prima di compilare, #include copia nel sorgente il contenuto dei file indicati*




Una riga che comincia con # è un'istruzione del precompilatore.

Il compilatore vero e proprio vede solo il file già «espanso».

# Le opzioni di gcc

*-Wall: «W» = warning, «all» = tutti: chiede di segnalare tutti gli avvisi*

| comando | cosa fa |
|---|---|
| `gcc -Wall -c prova.c` | compila prova.c e crea prova.o: nessun eseguibile |
| `gcc -Wall -o app.exe prova.c` | compila e linka: crea app.exe |
| `gcc -Wall -o app.exe prova.c -lm` | come sopra, usando la libreria matematica |
| `gcc -Wall -o app.exe prova.c grafica.o` | usa anche grafica.o, compilato in precedenza |
| `gcc -Wall -o app.exe prova.c grafica.c` | compila due sorgenti e crea app.exe |

*Leggete sempre gli avvisi: segnalano gli errori prima che il programma sbagli.*

# Stampare con printf

*La stringa contiene dei descrittori che indicano il tipo di ogni valore*

| descrittore | tipo |
|---|---|
| `%f, %e, %g` | float, double |
| `%d, %i` | int |
| `%c` | char (singolo) |
| `%s` | char (stringa) |

printf_float.c

```
float a = 1.2;
printf("a = %lf", a);
```

descrittore sbagliato: a è float

```
printf("a = %d", a);

a = 1073741824
```

```
stampa:  a = 1.200000
```

# Leggere con scanf

*Ogni tipo ha il suo descrittore; la variabile si indica con &*

| descrittore | tipo |
|---|---|
| `%f` | float |
| `%lf` | double |
| `%Lf, %llf` | long double |
| `%d, %i` | int |
| `%u` | unsigned int |
| `%c` | char (singolo) |
| `%s` | char (stringa) |

scanf

```
double angolo;
scanf("%lf", &angolo);
```

### %lf

lf = long float, cioè double

### &angolo

e commerciale: l'indirizzo della variabile angolo

# Esempio: da gradi a radianti

*Chiedere un angolo all'utente, convertirlo e stamparlo*

convert.c

```c
#include <math.h>
#include <stdio.h>

int main() {
    printf("inserisci angolo in gradi: ");
    double angolo;
    scanf("%lf", &angolo);

    double radianti;
    radianti = (angolo / 180.) * M_PI;
    printf("gradi: %lf, rad: %lf\n", angolo, radianti);
}
```

1. chiedere l'angolo all'utente
2. prendere il valore in gradi
3. convertire in radianti
4. stamparlo sullo schermo

Terminale

```
$ gcc -o app.exe convert.c -lm
$ ./app.exe
inserisci angolo in gradi: 180
gradi: 180.000000,
rad: 3.141593
```

# Dividere due interi

*In C il tipo del risultato dipende dal tipo degli operandi*

| espressione | operandi | risultato |
|---|---|---|
| `3 / 4` | intero / intero | `0` |
| `3. / 4` | reale / intero → reale / reale | `0.75` |
| `3 / 4.` | intero / reale → reale / reale | `0.75` |
| `3. / 4.` | reale / reale | `0.75` |

divisioni.c

```
double d1 = 3 / 4;
double d2 = 3. / 4;

d1 = 0.000000    d2 = 0.750000
```

## Regola

se i due operandi sono interi la divisione è intera e il resto si perde; se uno è reale, l'altro viene convertito

# Il cast: forzare il tipo

*int a = 2, b = 3;  e vogliamo 2/3 = 0.666…*

| espressione | cosa succede | risultato |
|---|---|---|
| `a / b` | intero / intero | `0` |
| `(double)a / b` | reale / intero → reale / reale | `0.666667` |
| `a / (double)b` | intero / reale → reale / reale | `0.666667` |
| `(double)a / (double)b` | reale / reale | `0.666667` |
| `(double)(a / b)` | prima la divisione tra interi | `0` |

### (double) a

casting: tratta a come un double

### Attenzione

il cast va fatto prima della divisione, non dopo

# Esercizi

*Da fare a casa, poi si controlla con un programma in C*

## 1 Area del cerchio

Scrivi un programma che legge il raggio r (double) e stampa area e circonferenza (M_PI).

## 2 Divisione

int a = 9, b = 4; double d = a / b; cosa stampa printf con %lf? Come ottenere 2.25?

## 3 Uguaglianza

int m = 8, n = 5; dopo n = m; quanto vale m == n ? E se al posto di n = m si scrive n == m; ?

## 4 Media

Scrivi un programma che legge due interi con scanf e stampa la loro media come numero reale.

# Riepilogo

| | | |
|---|---|---|
| 1 | **Uguaglianza** | = assegna, == confronta; il risultato di un confronto è 0 o 1 |
| 2 | **Funzioni e librerie** | sqrt e le altre funzioni di math.h; #include e compilare con -lm |
| 3 | **Compilazione** | il preprocessore espande #include; gcc -Wall segnala gli avvisi |
| 4 | **Input e output** | printf e scanf con i descrittori (%d, %lf …); &variabile in scanf |
| 5 | **Divisioni e cast** | tra interi il risultato è intero: usare un operando reale o il cast (double) |